

Agile Software Development

Sunday, October 12, 2025 9:42 AM

Agile Software Development

Agile means **developing software step-by-step** (in small parts) instead of building everything at once.

You divide the big project into **small tasks called “sprints”** (usually 1–2 weeks each).

After every sprint, you **test and show** what’s done, **get feedback**, and **improve** in the next round.

So,

Agile development is a **flexible and fast software development method** where projects are built step by step (**iteratively and incrementally**).

In agile, the team and customers **work together continuously**, adapting to new requirements and feedback.

It focuses on **quick delivery, team collaboration, and handling change easily**.

Example:

If a company is developing a **mobile banking app**, the agile team might first release only the **login and balance check feature**. Then, in the next update, they add **money transfer**, and later, **bill payment** — each version improving based on user feedback.

How Agile is different

- You **don’t wait till the end** to test the product — you test as you go.
- **Customer feedback** is taken after every small part.
- If any **mistake or change** is needed, it’s fixed early.
- Teams **communicate daily** and decide together.
- **Less paperwork**, more practical work and testing.
- **Faster delivery** — user gets working software early.

Agile Principles

1. Customer Involvement:

Customers work closely with the developers during all stages.

Example: The bank manager tests each new version of the banking app and gives feedback.

2. Incremental Delivery:

Software is developed and delivered in small parts (increments).

Example: First month – basic features, next month – add transaction history.

3. People, Not Process:

Focus on the team’s creativity and communication, not just following strict rules.

Example: Developers discuss ideas freely instead of waiting for management approval.

4. Embrace Change:

Agile welcomes changes, even if they come late.

Example: If users want fingerprint login instead of a password, it’s added in the next update.

5. Maintain Simplicity:

Keep the design simple and remove unnecessary features.

Example: Avoid adding too many buttons or complex menus in the app.

Advantages of Agile Method

- Rapid and continuous delivery keeps **customers satisfied**.
- **Direct communication** (meetings, discussions) instead of long documents.

- Can **adapt easily** to changing customer needs.
- Delivers **usable software early**, not at the very end.

Disadvantages of Agile Method

- Lack of **formal documentation** can cause confusion later.
- Harder to **maintain** the system when details aren't properly recorded.
- Can go off-track if the **customer's vision isn't clear**.
- Requires **experienced developers** who can make quick decisions.

When to Use Agile Method

- When **frequent changes** are expected during development.
- When **new features** need to be added or removed easily, based on user feedback.
- When **planning time is limited** — we can start quickly and adjust as we go.
- When **customer feedback is important** after each small part (iteration).
- When **requirements are not clear** at the beginning but will evolve later.

Example:

Creating a mobile app where users may suggest design or feature changes frequently.

Plan-Driven vs. Agile Development

Plan-Driven Development

All steps (like planning, design, coding, testing) are **decided early** before development starts.

Progress is measured **according to the original plan**.

Used when we have **detailed specification and clear design**.

Usually needs **less skilled developers** since tasks are well-defined.

Best for **large, stable systems** (like banking software).

Agile Development

Planning is **incremental** — we plan and adjust step-by-step.

Progress depends on **feedback and flexible updates**.

Used when **requirements are uncertain** or may **change often**.

Needs **more skilled and adaptive developers**.

Best for **small to medium systems** that need **quick updates** (like apps).

Agile Development Techniques

One of the most important agile approaches is called **Extreme Programming (XP)**.

It was created to make software development **faster, more flexible, and teamwork-based**.

In XP, developers work in **small steps (iterations)**, take **constant feedback**, and always keep the customer involved.

New code is **continuously tested and improved** — not left for the end.

Extreme Programming (XP)

Extreme Programming (XP) is an **Agile software development method** that focuses on **writing high-quality code** and **responding quickly to changing customer requirements**.

It's called "*Extreme*" because it takes good software practices (like testing, feedback, and communication) to an **extreme level** — meaning it's done **more frequently and strictly** than usual.

XP is about:

- **Frequent communication** between developers and customers.
- **Continuous testing and improvement** of the code.
- **Delivering software in small, quick updates** instead of one big release.
- **Teamwork and flexibility** rather than strict rules and documents.

When XP is Used:

XP works best when:

- Requirements **change very frequently**.
- The **customer can stay in close contact** with the development team.
- The project is **small to medium-sized**.
- The team members are **skilled and experienced** developers.

Example:

A startup is building a new mobile app where users can order groceries.

Since customer needs change often (new features, new design ideas), XP helps by allowing developers to make small, frequent updates and test them right away.

Principle / Practice	Simple Explanation
Collective Ownership	Every developer can change or improve any part of the code. It's not "my code" or "your code" — it belongs to everyone. <i>Example:</i> If one developer sees a mistake made by another, they can fix it directly.*
Continuous Integration	When a small part of code is completed, it is immediately added to the main project, so errors are found early. <i>Example:</i> After finishing a login feature, it's added and tested with the main app immediately.*
Incremental Planning	All requirements are written as short "user stories" on cards and developed step by step. <i>Example:</i> A story card might say: "As a user, I want to reset my password."*
On-site Customer	The customer (or their representative) stays close to the team and gives feedback every day. <i>Example:</i> A client sits with developers to test new features and suggest small changes instantly.*
Pair Programming	Two programmers work together on one computer — one writes code, the other reviews it. <i>Example:</i> One types, the other checks for logic errors or suggests improvements.*
Simple Design	Only design what is needed for current tasks — no unnecessary features. <i>Example:</i> Build a simple login system first, add social login later only if required.*

Agile Project Management

In traditional (plan-based) project management, a manager makes a **complete plan before starting** the project.

Everything — like time, cost, and features — is fixed in advance.

But in real life, **requirements often change** as customers realize new needs.

This makes strict planning difficult.

That's why **Agile Project Management** is used.

It focuses on **delivering small parts of the software step by step**, instead of waiting for the whole project to finish.

Each small part (called an *increment*) is completed quickly, tested, and shown to the customer for feedback.

This way:

- The project can **adapt easily to new changes**.
- Customers **see progress regularly**.
- Developers **get feedback early** and improve continuously.

Example:

If a company is building an e-commerce app, the first increment may include “user login,” the next may add “product listing,” and later increments add “payment” and “order tracking.”

Scrum Framework

Scrum is one of the most popular agile methods.

It is an **iterative and incremental framework** — meaning work is done in **repeated cycles (iterations)** and **small deliverable parts (increments)**.

In Scrum:

- The project is divided into **small teams** (usually 5–9 members).
- Each team works together closely and completes work in short cycles called **Sprints** (usually 2–4 weeks).
- After each sprint, a working version of the software is delivered to the customer.

Example:

A Scrum team developing a school management system may complete:

- Sprint 1 → Student login & registration
- Sprint 2 → Teacher module
- Sprint 3 → Fee payment system

Each sprint adds a working feature, building up the final system gradually.